

Spec-Driven Development with Supervised RAG Refinement: A Framework for AI-Assisted Software Development in Operational Weather Forecasting

NOAA EMC Global Workflow MCP Team
Environmental Modeling Center
National Oceanic and Atmospheric Administration
Terry.McGuinness@noaa.gov

November 19, 2025

Abstract

We present the **Spec-Driven Development (SDD) Framework**, a novel methodology for AI-assisted software development that combines structured workflow specifications with hybrid graph-enhanced retrieval-augmented generation (RAG) and supervised Subject Matter Expert (SME) refinement. The framework enables systematic documentation, validation, and execution of complex software development tasks through machine-readable specifications in YAML and Markdown formats.

A key innovation is the integration of semantic annotations in reStructuredText (RST) documentation that preserve human-readable content while embedding machine-processable metadata for enhanced AI comprehension. We demonstrate the framework’s effectiveness through deployment in the NOAA Global Workflow system, achieving autonomous code analysis, compliance validation, and development task execution.

The SME Review Guide mechanism enables domain experts to refine AI knowledge bases through structured annotation reviews, improving retrieval accuracy by $3\text{--}5\times$ compared to pure text-based search. Empirical evaluation with 100 developer queries shows Precision@10 improvement from 0.23 (baseline) to 0.87 (hybrid approach), with development velocity gains of $5\text{--}9\times$ for complex tasks. This work addresses critical challenges in maintaining large-scale operational software systems where domain expertise is scarce and documentation complexity exceeds human cognitive capacity.

Keywords: Spec-Driven Development, Retrieval-Augmented Generation, Knowledge Graphs, Software Development Automation, Subject Matter Expert Refinement, Semantic Annotations, Weather Forecasting Systems

1 Introduction

1.1 Motivation

Modern operational weather forecasting systems, such as NOAA’s Global Workflow (GFS/GEFS), comprise millions of lines of code spanning multiple programming languages, hardware architectures, and scientific domains. The Global Workflow alone integrates:

- **Unified Forecast System (UFS)** atmospheric model ($\sim 500\text{K}$ LOC Fortran)

- **GSI/GDAS** data assimilation system (~300K LOC)
- **Workflow orchestration** (Rocoto XML, 7,000+ task dependencies)
- **Python workflow execution** (wxflow library)
- **Shell scripting** (2,000+ operational scripts)
- **HPC-specific configurations** across 5 platforms

This complexity creates fundamental challenges:

1. **Knowledge Scarcity:** Few individuals understand the complete system
2. **Documentation Decay:** Code evolves faster than documentation updates
3. **Onboarding Friction:** New developers require 6–12 months to become productive
4. **Compliance Burden:** NCEP Central Operations mandates (EE2 standards) require manual auditing
5. **Operational Risk:** Silent failures can cascade through 6-hour forecast windows

1.2 The SDD Framework Approach

The **Spec-Driven Development (SDD) Framework** addresses these challenges through three core innovations:

1. **Machine-Readable Workflow Specifications:** Development tasks are expressed as structured workflows in YAML/Markdown that AI agents can parse, validate, and execute.
2. **Hybrid Graph-Enhanced RAG:** Knowledge bases combine vector embeddings (semantic search) with Neo4j graph databases (code structure).
3. **Supervised SME Refinement:** Subject matter experts review and refine semantic annotations embedded in RST documentation through a systematic review guide.

1.3 Contributions

This paper makes the following contributions:

1. **SDD Framework Specification:** A complete methodology for AI-assisted software development with formal workflow syntax (Section 2)
2. **Semantic Annotation Schema:** MCP directive system for embedding machine-processable metadata in human-readable documentation (Section 3)
3. **SME Review Guide:** Structured methodology for domain experts to refine AI knowledge bases (Section 4)
4. **Hybrid RAG Architecture:** Graph-enriched retrieval combining vector search with code structure analysis (Section 5)
5. **Empirical Validation:** Deployment in NOAA Global Workflow with measured improvements (Section 6)
6. **YAML Task Tracking:** Specification format for complex multi-phase development projects (Section 7)

2 Spec-Driven Development Framework

2.1 Core Principles

The SDD Framework is built on four foundational principles:

P1: Specifications as First-Class Artifacts. Development tasks are expressed as machine-readable specifications before implementation. Specifications define: (1) What the system should do, (2) How to validate success, (3) When tasks should execute, (4) Who is responsible.

P2: Progressive Refinement. Specifications evolve from high-level intent to detailed implementation plans through iterative refinement.

P3: Health-Integrated Architecture. All components integrate health monitoring from initialization. System health is a first-class concern.

P4: Supervised Human-AI Collaboration. AI agents propose implementations based on specifications. Human experts review, refine, and approve.

2.2 Workflow Execution Model

SDD workflows are executed through the following process:

Algorithm 1 SDD Workflow Execution

- 1: Parse workflow specification $\rightarrow$ structured workflow object
 - 2: Validate: Check required fields, consistent dependencies
 - 3: Plan: Topological sort of tasks by dependencies
 - 4: **for** each step s in execution plan **do**
 - 5: **Health Check:** Verify system/data availability
 - 6: **Data Query:** Retrieve required context from RAG
 - 7: **Validation:** Verify inputs meet criteria
 - 8: **Command Execution:** Run system commands (if applicable)
 - 9: **Ingestion:** Update knowledge base (if code changed)
 - 10: **end for**
 - 11: Aggregate results, metrics, health status
 - 12: Record execution history with timestamps
-

3 Semantic Annotation Schema

3.1 The Documentation Comprehension Problem

Traditional documentation presents a fundamental challenge for AI systems: text is optimized for human interpretation, not machine processing. Consider this requirement from NCEP standards:

“The utilities listed below **must** be used to assist in accomplishing certain tasks for all WCOSS models.”

A human reader understands: (1) “must” = mandatory requirement, (2) “all WCOSS models” = global scope, (3) these are helper utilities.

An AI text search sees: Keywords (“utilities”, “must”, “WCOSS models”) with no structured priority level, no enforcement mechanism, no operational rationale.

3.2 MCP Directive System

The **Model Context Protocol (MCP) Directive System** embeds semantic metadata directly in RST documentation using custom directives. Table 1 summarizes the seven core directive types.

Table 1: MCP Directive Types

Directive	Purpose	Key Fields
<code>mcp:compliance</code>	Priority/scope	priority, type, scope
<code>mcp:intent</code>	Why it exists	description, enforcement, rationale
<code>mcp:severity</code>	RFC 2119 level	must/should/may, exceptions
<code>mcp:utility</code>	Tool metadata	module, category, required
<code>mcp:example</code>	Code examples	language, context, demonstrates
<code>mcp:pattern</code>	Design patterns	category, anti-pattern, alternatives
<code>mcp:see-also</code>	Relationships	related, type (prerequisite/alternative)

Example annotation:

```

1 .. mcp:intent:: rapid_error_detection
2   :description: Enable immediate error detection
3   :enforcement: runtime_check
4   :rationale: 99% on-time delivery SLA requires
5               detection within 5 minutes
6
7 .. mcp:utility:: err_chk
8   :module: prod_util
9   :category: error-handling
10  :required: yes

```

Listing 1: Semantic Annotation Example

3.3 Annotation Benefits

Semantic annotations provide three critical capabilities:

1. Intent-Aware Search. Query: “How do I check for errors?”

Before (Text Search): Returns all text containing “error” (hundreds of hits), no priority information.

After (Intent-Aware): Filters to `mcp:intent::rapid_error_detection`, shows `err_chk` with `priority: critical`, includes rationale and usage examples.

2. Compliance Validation. AI understands code implements `error_check_pattern` intent but flags missing `FATAL ERROR:` prefix.

3. Relationship Traversal. Starting from `err_chk`: Graph shows code files calling it, annotations show related utilities, examples show usage patterns.

4 SME Review Guide: Supervised RAG Refinement

4.1 Review Methodology

The SME Review Guide structures expert feedback across seven dimensions:

- 1. Intent Accuracy:** Validate `:description:` and `:rationale:` capture true operational purpose

2. **Priority/Severity:** Ensure priority reflects real operational impact
3. **Utility Metadata:** Verify module names, categories, required/deprecated status
4. **Code Examples:** Validate examples demonstrate correct usage and cover common scenarios
5. **Relationships:** Verify `:related:` items and relationship types
6. **Patterns:** Identify design patterns and anti-patterns for recognition
7. **Gap Analysis:** Identify missing annotations, examples, relationships

4.2 Review Workflow

1. **Pilot Annotation:** Development team annotates 1–2 sections
2. **SME Review:** Domain experts provide structured feedback
3. **Iteration:** Incorporate feedback, refine patterns
4. **Validation:** Test retrieval accuracy improvements
5. **Production:** Full documentation annotation and deployment

4.3 Measured Impact

Table 2 shows measured improvements from SME-refined annotations in NOAA Global Workflow.

Table 2: SME Refinement Impact			
Metric	Before	After	Improvement
Retrieval Precision	45%	87%	+93%
False Positives (Compliance)	35%	8%	−77%
Query Resolution Time	2–5 min	15–30 sec	−80%
New Developer Onboarding	8 weeks	3 weeks	−63%

5 Hybrid Graph-Enhanced RAG Architecture

5.1 Graph-Enhanced Retrieval

The SDD Framework integrates Neo4j graph database with ChromaDB vector store:

- **ChromaDB:** Semantic similarity search (768-dimensional embeddings)
- **Neo4j:** Code structure and relationships (78,339+ relationships)

When ingesting code, we enrich vector metadata with graph context:

```

1 metadata = {
2     "file_path": "scripts/exglobal_forecast.py",
3     "content_type": "code",
4     "functions_defined": ["run_forecast", "init_model"],
5     "calls_functions": ["err_chk", "prep_step"],
6     "imports_modules": ["wxflow", "numpy"],
7     "called_by_files": ["rocoto/forecast_job.xml"]
8 }

```

Listing 2: Graph-Enriched Metadata

5.2 Hybrid Query Algorithm

Algorithm 2 Hybrid RAG Query

- 1: Vector search for semantic similarity (top 50)
 - 2: Extract entities mentioned in query
 - 3: **for** each entity e **do**
 - 4: Graph traversal for structural context
 - 5: Find callers, callees, dependencies
 - 6: **end for**
 - 7: Merge vector and graph results
 - 8: Rank by: $\alpha \cdot \text{cosine_sim} + \beta \cdot \text{graph_rel} + \gamma \cdot \text{annot_match}$
 - 9: **return** top k results
-

Where α, β, γ are tunable weights (default: 0.5, 0.3, 0.2).

5.3 Query Type Optimization

Different query types benefit from different balance of vector/graph/annotation. Table 3 shows optimized weights.

Table 3: Query Type Weight Optimization

Query Type	Vector	Graph	Annot	Example
Concept	0.7	0.1	0.2	What is Rocoto?
Usage	0.5	0.3	0.2	How do I use err_chk?
Code Structure	0.2	0.7	0.1	What calls this function?
Compliance	0.3	0.2	0.5	Is this EE2 compliant?
Troubleshooting	0.4	0.4	0.2	Why is this failing?

6 Empirical Evaluation

6.1 Deployment Architecture

Target System: NOAA Global Workflow (GFS/GEFS operational forecasting)

Infrastructure:

- HPC Platforms: Hera, Hercules, Orion (RDHPCS), WCOSS2 (NCEP Ops)
- Compute: 8-core VM with 32GB RAM
- Storage: 100GB for knowledge bases (5,307 documents, 78,339 relationships)

6.2 Retrieval Accuracy Evaluation

Methodology: 100 queries from actual developer questions, expert-labeled ground truth.

Metrics:

- Precision@10: Fraction of top-10 results that are relevant
- MRR (Mean Reciprocal Rank): $\frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$
- Recall@50: Fraction of relevant documents in top 50

Table 4 shows retrieval accuracy across approaches.

Table 4: Retrieval Accuracy Comparison

Approach	P@10	MRR	R@50
Text Search (Baseline)	0.23	0.31	0.48
Pure Vector (MPNet)	0.45	0.52	0.71
Vector + Annotations	0.67	0.73	0.84
Hybrid (All)	0.87	0.91	0.95

Finding: Hybrid approach achieves $3.8\times$ improvement in Precision@10 over baseline.

6.3 Development Velocity Impact

Case Study: GitHub Issue #4220 - C48 ATM CTest Path Mapping

- **Before SDD (Estimated):** 8–13 hours
 - Manual code archaeology: 4–6 hours
 - Documentation writing: 2–3 hours
 - Review cycles: 2–4 hours
- **With SDD (Actual):** 1.5 hours
 - RAG-assisted code analysis: 45 minutes
 - Spec-driven documentation: 30 minutes
 - Automated validation: 15 minutes

Improvement: $5.3\text{--}8.7\times$ faster with higher accuracy.

7 YAML Task Tracking

For multi-week development projects, the SDD Framework supports comprehensive YAML specifications that capture:

- Project metadata (version, author, GitHub issue, repository/branch)
- Project overview (objective, scope, estimated duration/effort)
- Timeline & phases (structured breakdown with task dependencies)
- Resource management (personnel, HPC resources, storage)
- Risk management (risk identification, probability, impact, mitigation)
- Success metrics (quantifiable completion criteria)

Example: EVS EE2 Compliance Remediation demonstrates a 12-week, 462-hour project to remediate 670 non-compliant files with structured phasing, resource allocation, and risk mitigation.

8 Lessons Learned

8.1 Technical Insights

1. Graph Enrichment is Non-Negotiable. Pure vector search fails for code intelligence. Graph relationships (CALLS, IMPORTS, DEFINES) provide structural context that semantic similarity cannot capture.

2. Annotation Quality > Annotation Quantity. 10 high-quality annotated sections with SME review provide better retrieval than 100 auto-generated annotations without validation.

3. Embedding Model Selection Matters. Evaluation of 5 models shows MPNet (768-dim) provides best quality/speed/cost trade-off: P@10=0.87, inference time=25ms.

4. Health Monitoring Must Be First-Class. Systems without health integration suffer from silent failures. Every tool should record success/failure metrics from initialization.

8.2 Operational Insights

1. Version Control for Knowledge Bases. Knowledge bases must stay synchronized with code. Automated CI/CD pipeline essential: trigger on commits, re-parse changed files, update graph incrementally.

2. Staged Rollout is Essential. Phased approach: Week 1 (pilot 2 sections), Week 2 (SME review), Week 3–6 (expand with validated patterns), Week 7+ (continuous refinement).

3. Developer Training Required. Demonstrate 3–5× speed improvements on real tasks, show case studies, provide AI-assisted vs. manual comparisons.

9 Related Work

Retrieval-Augmented Generation: Lewis et al. [1] introduced RAG for open-domain QA. Our work extends RAG to code intelligence with graph enrichment and supervised SME refinement.

Code Intelligence: GitHub Copilot [2] generates code from natural language. Our system *explains* existing code using hybrid retrieval. Complementary approaches.

Knowledge Graph Construction: ERNIE [3] integrates knowledge graphs with pre-trained models. Our graph schema is code-specific (CALLS, IMPORTS, DEFINES).

10 Conclusion

The **Spec-Driven Development (SDD) Framework** demonstrates that AI-assisted software development can achieve human-expert-level code intelligence through three innovations:

1. Machine-readable workflow specifications enable systematic task decomposition
2. Semantic annotations bridge the gap between human-readable text and machine-processable metadata
3. Supervised SME refinement systematically captures domain expertise

Deployment in NOAA Global Workflow validates the approach with empirical improvements: 3.8× retrieval accuracy, 5–9× development velocity, 77% false positive reduction.

Key Takeaway: AI assistance is most effective when domain experts refine the knowledge base through structured methodologies like the SME Review Guide. Pure automation fails—supervised human-AI collaboration succeeds.

References

- [1] Lewis, P., et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.
- [2] Chen, M., et al. *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374, 2021.
- [3] Zhang, Z., et al. *ERNIE: Enhanced Language Representation with Informative Entities*. ACL 2019.
- [4] NOAA Environmental Modeling Center. *NCEP Central Operations Implementation Standards (EE2)*. <https://nws-hpc-standards.readthedocs.io/>, 2025.
- [5] NOAA Global Workflow Development Team. *Global Workflow Documentation*. <https://github.com/ufs-community/global-workflow>, 2025.